



التوثيق التقني الشامل

Code Craft Core

منصة التعلم التفاعلية والألعاب البرمجية — بيئة تعليمية عربية متكاملة لتعلم البرمجة بأسلوب الألعاب والمكافآت.

Supabase

Prisma + PostgreSQL

TypeScript

React 19

Next.js 16

Tailwind CSS v4

نوع الوثيقة: توثيق معماري وتقني (الأدوات، المخططات، الأمان).
الإصدار: 0.1.0 ▪ اللغة: العربية (RTL)

جدول المحتويات



دليل التنقل بين أقسام الوثيقة التقنية.

الرؤية والوحدات الأساسية	1	نظرة عامة على المشروع
جدول تفصيلي لكل أداة واستخدامها	2	الأدوات والتقنيات المستخدمة
النمط المعماري والطبقات وتدفق الطلب	3	معمارية النظام
Client / Server وفصل المسؤوليات	4	طبقة العميل وطبقة الخادم
الكيانات والعلاقات الأساسية	5	مخطط قاعدة البيانات (ERD)
النماذج والخدمات والعلاقات	6	مخطط الأصناف (Class Diagram)
الفاعلون والعلاقات «include» و«extend»	7	مخطط حالات الاستخدام (Include / Extend)
المصادقة، الصلاحيات، والحماية	8	أمن البيانات وسلامتها ونزاهتها

1 نظرة عامة على المشروع

Code Craft Core منصة تعليمية عربية لتعلّم البرمجة تعتمد على التلعيب (Gamification) ونظام اقتصادي داخلي (نقاط XP وعملات Craft Coins) لتحفيز المتعلمين.

• الرؤية

توفير رحلة تعلّم برمجي متكاملة باللغة العربية، من المستوى المبتدئ إلى الاحتراف، عبر مسارات منّظمة، ودروس فيديو ونصية، واختبارات وتحديات وألعاب برمجية، مع نظام معلمين لإنشاء وإدارة المحتوى ولوحة إدارة مركزية.

• الوحدات الأساسية للنظام

التقييم والاختبارات

اختبارات (Quizzes)، بنك أسئلة، محاولات، واختبارات تحديد المستوى المدعومة بالذكاء الاصطناعي.

التعلّم والمحتوى

مسارات تعلّم (Tracks)، مقررات، وحدات، دروس، مصادر، ومقالات مع محرر نصوص غني.

منظومة المعلمين

طلبات الانضمام، ملفات المعلمين، سير الموافقات (Approvals)، والمحافظ والمدفوعات.

التلعيب والاقتصاد

نقاط الخبرة XP، عملات Craft Coins، الشارات، الإنجازات، لوحة الصدارة، والسلاسل اليومية (Streaks).

الذكاء الاصطناعي

مساعد ذكي، توصيات، بطاقات تعليمية، توليد اختبارات، تلخيص، وتحليلات تعلّم.

المجتمع والتفاعل

منتديات، أسئلة وأجوبة، تعليقات، تفاعلات، متابعة، رسائل خاصة، وبلاغات.

الإدارة والأمان

نظام صلاحيات RBAC، سجلات تدقيق، إعدادات النظام، وتحديد معدّل الطلبات.

تحقيق الدخل

اشتراكات، مدفوعات، فواتير، مشتريات، كوبونات، عروض، وبرنامج تسويق بالعمولة.

تحتوي قاعدة البيانات على أكثر من 80 نموذجاً (Model) موزّعة على هذه الوحدات، ما يعكس اتساع نطاق المنصة وتكاملها.

2 الأدوات والتقنيات المستخدمة

جميع المكتبات والأدوات الرئيسية المعتمدة في المشروع مع بيان الغرض من كل منها (مستخلصة من ملف `package.json`).

● الإطار والواجهة الأمامية (Framework & Frontend)

الأداة	الاستخدام في المشروع
Next.js 16	الإطار الأساسي باستخدام App Router — تصيير الصفحات على الخادم (RSC)، مسارات API، و Server Actions.
React 19	مكتبة بناء واجهات المستخدم والمكوّنات التفاعلية (Client Components).
TypeScript 5	لغة البرمجة الأساسية لضمان أمان الأنواع (Type Safety) عبر كامل المشروع.
Tailwind CSS v4	نظام التنسيق (Utility-first)، مع <code>@tailwindcss/typography</code> لتنسيق المحتوى الغني.
next-themes	إدارة الوضع الليلي/النهارى (Dark / Light Mode).
lucide-react	مكتبة الأيقونات المتّسقة عبر الواجهة.

● المحرّرات والمحتوى

الأداة	الاستخدام في المشروع
@monaco-editor/react	محرّر الأكواد (Code Playground) لكتابة وتجربة البرمجة داخل المنصة.
@tiptap/* (React)	محرّر نصوص غني (Rich Text) للمقالات والدروس: صور، روابط، محاذاة، تسطير، وعناصر نائبية.
jspdf + html2canvas	توليد شهادات الإتمام (Certificates) كملفات PDF من داخل المتصفح.

الأدوات والتقنيات المستخدمة

2

تابع — الخادم، قاعدة البيانات، المصادقة، الذكاء الاصطناعي، والتحقق والاختبار.

الخادم وقاعدة البيانات (Backend & Database)

الأداة	الاستخدام في المشروع
Prisma 7 (@prisma/client)	الـ ORM للتعامل مع قاعدة البيانات، تعريف المخطط (Schema)، والترحيلات (Migrations).
@prisma/adapter-pg + pg	محول الاتصال بقاعدة PostgreSQL عبر تجعّع الاتصالات (Connection Pooling).
PostgreSQL	قاعدة البيانات العلائقية الأساسية لتخزين كل بيانات المنصة.
Supabase (@supabase/supabase-js, @supabase/ssr)	خدمة المصادقة (Auth)، التخزين (Storage)، وإدارة الجلسات على الخادم والعميل.
@aws-sdk/client-s3	رفع وتخزين الملفات والوسائط (متوافق مع تخزين S3).
sharp	معالجة وضغط الصور على الخادم لتحسين الأداء.

المصادقة والأمان

الأداة	الاستخدام في المشروع
bcrypt	تجزئة (Hashing) كلمات المرور بأمان قبل تخزينها في قاعدة البيانات.
zod	التحقق من صحة المدخلات (Input Validation) وتعريف مخططات البيانات.
react-hook-form + @hookform/resolvers	إدارة النماذج والتحقق منها في الواجهة مع ربطها بمخططات Zod.

الذكاء الاصطناعي

الأداة	الاستخدام في المشروع
@google/generative-ai	نماذج Google Gemini للمساعد الذكي والتوصيات وتوليد الاختبارات والتلخيص.
groq-sdk	استدلال سريع للنماذج اللغوية عبر Groq لميزات الذكاء الاصطناعي.

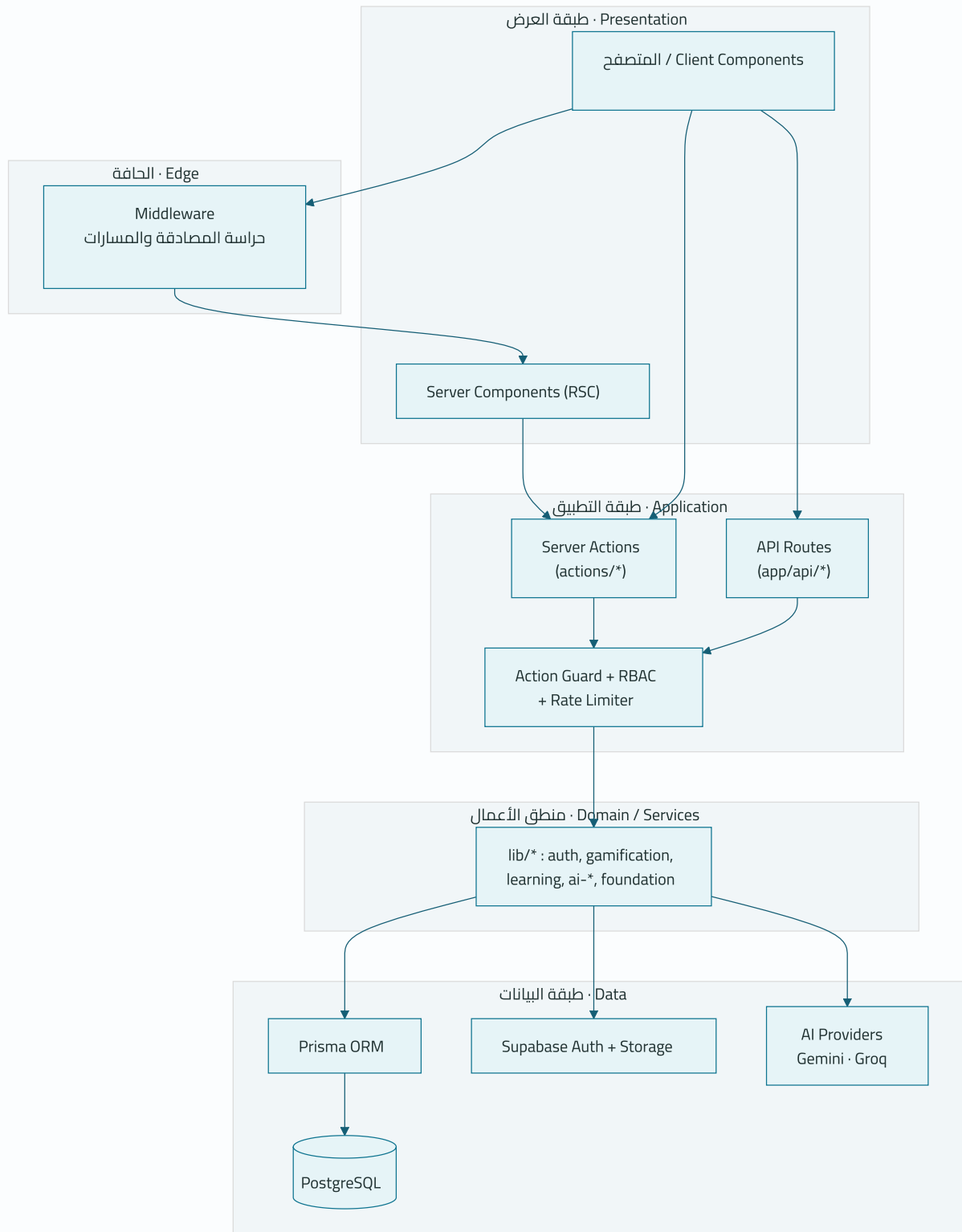
الأدوات المساعدة والاختبار والنشر

الأداة	الاستخدام في المشروع
date-fns	تنسيق ومعالجة التواريخ.
Vitest + @vitejs/plugin-react	إطار اختبارات الوحدة (Unit Testing) — مثل اختبار محدّد معدّل الطلبات.
ESLint + eslint-config-next	فحص جودة الكود والالتزام بالمعايير.
tsx	تشغيل سكريبتات TypeScript مباشرة (مثل البذور Seed).
Docker	حاوية النشر (standalone) لتشغيل المنصة في أي بيئة.

يعتمد المشروع على معمارية موحّدة قائمة على Next.js App Router تدمج الواجهة والخادم في تطبيق واحد (Full-Stack) مع فصل واضح للمسؤوليات عبر الطبقات.

● النمط المعماري

يتبع النظام نمطاً متعدّد الطبقات (Layered Architecture) يفصل بين العرض، ومنطق الأعمال، والوصول للبيانات، مع الاعتماد على مكوّنات الخادم في **React (RSC)** و **Server Actions** كطبقة تطبيق آمنة، و **Prisma** كطبقة وصول للبيانات، و **PostgreSQL/Supabase** كطبقة تخزين.



شكل 3-1: مخطط المعمارية العام وتدفق الطلب عبر الطبقات.

3 معمارية النظام

تفصيل طبقات النظام ومسؤولية كل طبقة.

(1) طبقة العرض (Presentation Layer)

مكونات (Server & Client) React في `app/` و `components/` — الصفحات، لوحات التحكم (طالب/معلم/مدير)، والنماذج التفاعلية.

(2) طبقة الحافة (Edge / Middleware)

`middleware.ts` يتحقق من جلسة Supabase، ويحرس مسارات `/dashboard`، ويلتقط كود الإحالة، ويعيد توجيه المستخدمين المصادق عليهم.

(3) طبقة التطبيق (Application Layer)

`actions/*` (Server Actions) و `app/api/*` (API Routes) — نقاط الدخول لمنطق الأعمال، محمية عبر `guardAction` و RBAC ومحدّد معدّل الطلبات.

(4) طبقة منطق الأعمال (Domain / Services)

`lib/*` — خدمات مثل المصادقة، التلعيب، التعلّم، الذكاء الاصطناعي، الحوكمة والهوية، والأساس (foundation).

(5) طبقة البيانات (Data Layer)

Prisma ORM فوق PostgreSQL، إضافة إلى Supabase (المصادقة والتخزين) ومزوّد الذكاء الاصطناعي (Gemini / Groq).

● مبادئ التصميم المعتمدة

- **فصل المسؤوليات:** كل طبقة معزولة ولا تتجاوز حدودها؛ الواجهة لا تصل للبيانات مباشرة بل عبر Server Actions.
- **الأمان أولاً بالتصميم:** كل عملية حساسة تمرّ عبر `guardAction` للتحقق من الجلسة والصلاحيّة والحد من التكرار.
- **الذرية (Atomicity):** العمليات **❖❖** مركّبة (كالتسجيل) تُنفّذ داخل معاملات Prisma (`$transaction`).
- **قابلية التوسّع:** فصل مزوّد الخدمات (Supabase / AI / S3) خلف وحدات مستقلة يسهّل استبدالها.
- **اختلاف البيانات:** إعدادات أكثر صرامة في الإنتاج (فحص الأنواع، رؤوس الأمان، الكاش، تحديد المعدّل).

طبقة العميل وطبقة الخادم

يفصل النظام بوضوح بين ما يُنفَّذ في متصفح المستخدم (Client) وما يُنفَّذ على الخادم (Server) لضمان الأمان والأداء.

طبقة الخادم (Server Side)

تعمل على الخادم وتحتكر منطق الأعمال والبيانات:

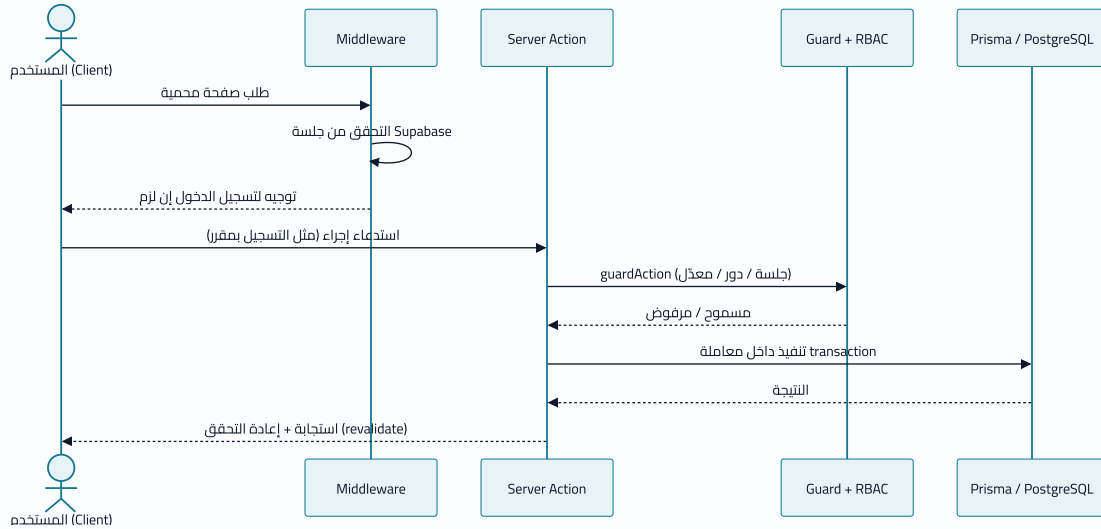
- Server Components لجلب البيانات مباشرة
- Server Actions ("use server") للعمليات
- API Routes للتكاملات وفحوص الصحة
- Prisma للوصول لقاعدة البيانات
- المصادقة، RBAC، والأسرار (Secrets)

طبقة العميل (Client Side)

تعمل في المتصفح وتُعنى بالتفاعل والعرض فقط:

- Client Components ("use client")
- النماذج التفاعلية عبر react-hook-form + Zod
- محرّرات Monaco و Tiptap والألعاب البرمجية
- إدارة الحالة والـ theming (next-themes)
- توليد الشهادات (jspdf/html2canvas) PDF

• تدفّق الطلب بين الطبقتين



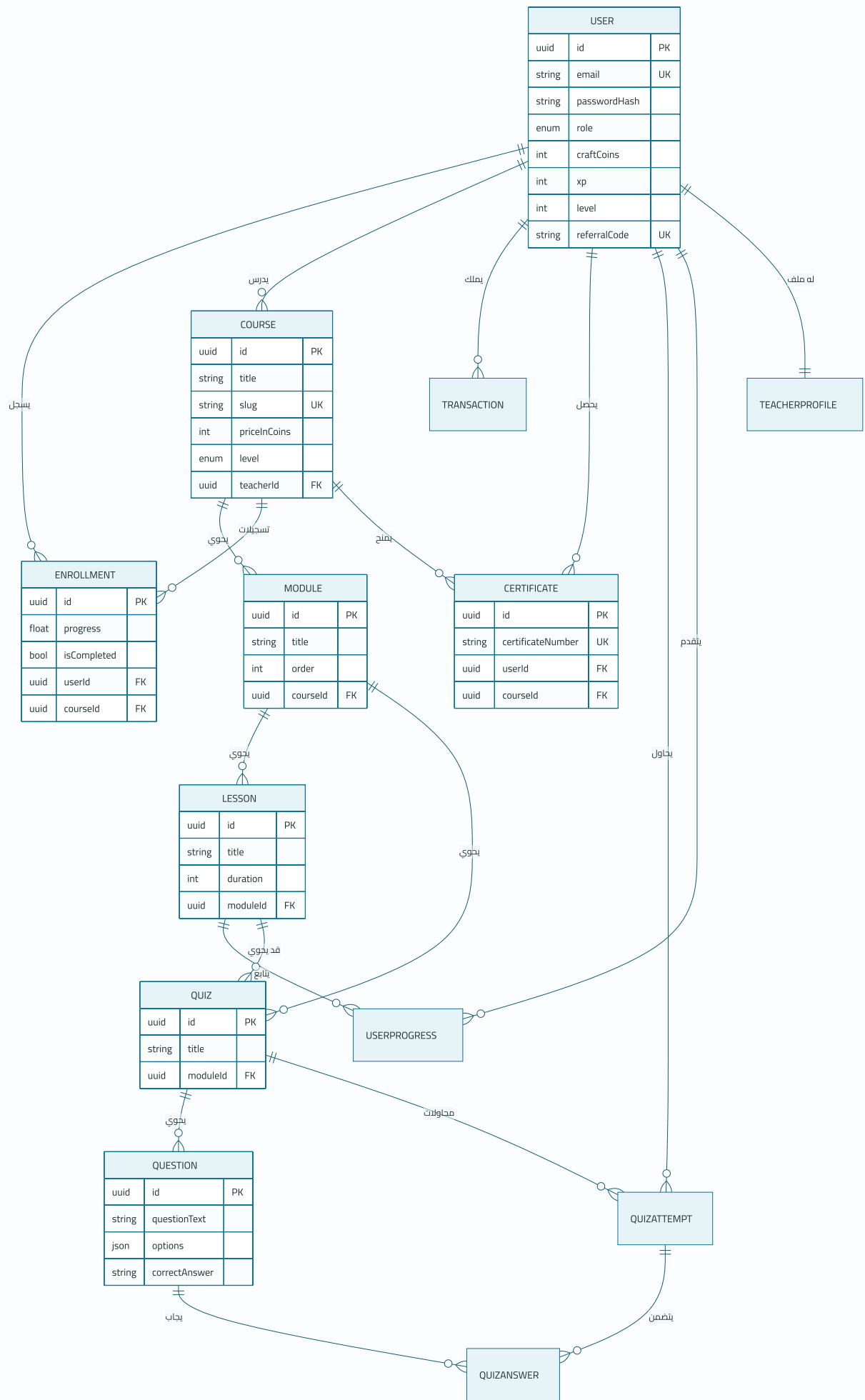
شكل 4-1: تسلسل التفاعل بين العميل والخادم عبر بوابة الحماية.

لا يصل العميل إلى قاعدة البيانات مطلقاً بشكل مباشر؛ كل وصول يمرّ عبر الخادم بعد اجتياز طبقة الحماية، ما يمنع تسريب الأسرار وتجاوز الصلاحيات.

مخطط قاعدة البيانات (ERD)

5

يوضح المخطط الكيانات الأساسية للنطاق التعليمي وعلاقاتها. (القاعدة تضم أكثر من 80 نموذجاً؛ هنا نُبرز النواة لضمان الوضوح.)



مخطط قاعدة البيانات (ERD) 5

شرح العلاقات ومجموعات النماذج.

• شرح العلاقات الأساسية

العلاقة	النوع	الوصف
User → Course	واحد لمتعدد	المعلم الواحد يُنشئ عدّة مقررات (TeacherCourses).
User ↔ Course	متعدد لمتعدد	عبر Enrollment — الطالب يسجل في عدّة مقررات والمقرر به عدّة طلاب.
Course → Module → Lesson	هرمي	المقرر يحوي وحدات، والوحدة تحوي دروساً مرتّبة.
Quiz → Question → QuizAnswer	هرمي	الاختبار يحوي أسئلة، وكل محاولة تُنتج إجابات مرتبطة بالأسئلة.
User → Certificate	واحد لمتعدد	الطالب يحصل على شهادة فريدة عند إتمام كل مقرر.
User → User	انعكاسية	نظام الإحالة (referrals / referredBy).

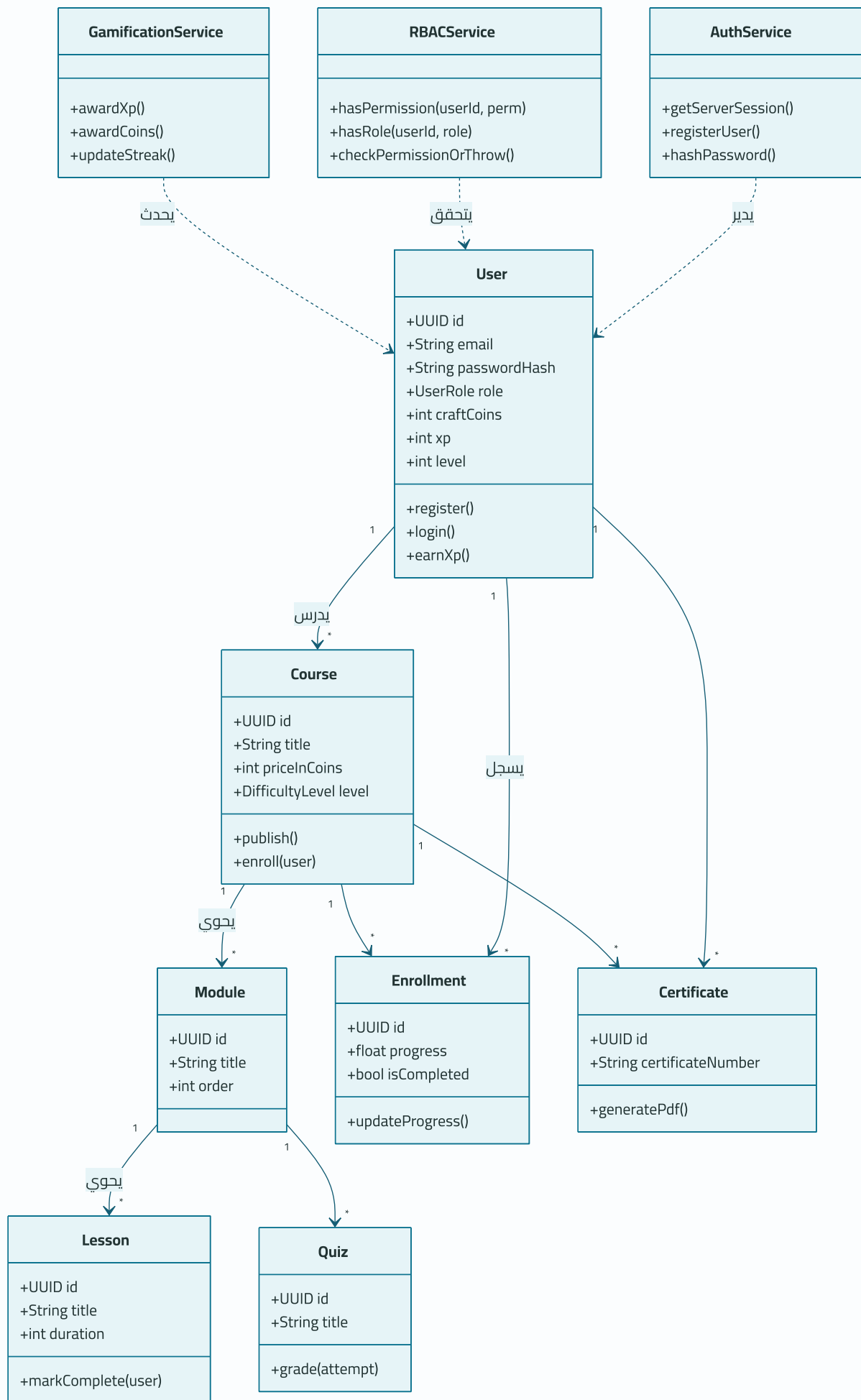
• مجموعات النماذج (Model Domains)

التعلم LearningTrack, Course, Module, Lesson, LessonResource, Article, LearningPath, UserProgress	الهوية والحوكمة User, Role, Permission, RolePermission, Session, EmailVerification, PasswordReset, AuditLog
المجتمع والدفع والذكاء ForumTopic, QaQuestion, Follow, Subscription, Payment, Invoice, AiConversation, AiRecommendation	التقييم والتلعيب Quiz, Question, QuizAttempt, GameResult, XpTransaction, Badge, UserAchievement, Certificate

مخطط الأصناف (Class Diagram)

6

يمثل الأصناف الرئيسية (النماذج وخدمات منطق الأعمال) وسماتها وعملياتها وعلاقاتها.

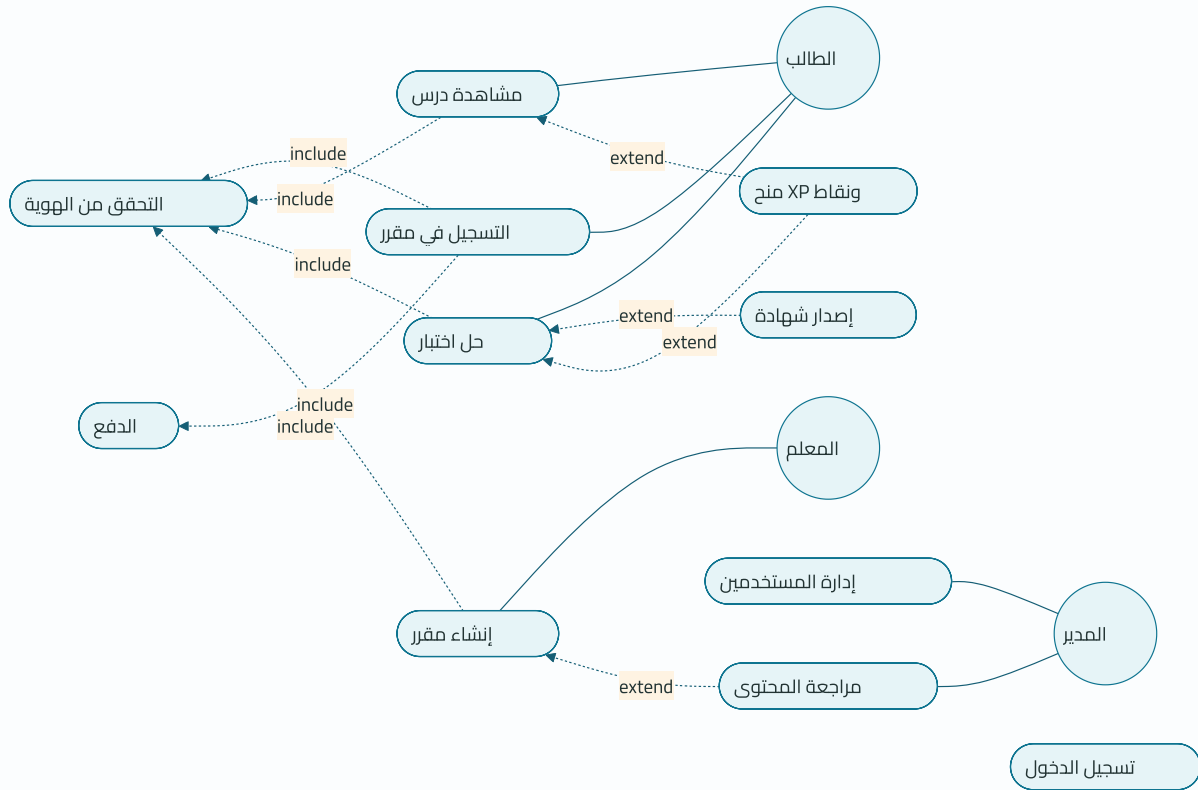


شكل 6-1: مخطط الأنصاف للنماذج الأساسية وخدمات منطق الأعمال.

الخدمات (`AuthService` , `RBACService` , `GamificationService`) تقابل وحدات في `lib/` مثل `auth.ts` , `rbac.ts` , و `gamification.ts` , وتعبر عن السلوك (العمليات) بينما تمثل النماذج البيانات (السمات).

7 مخطط حالات الاستخدام

يوضح الفاعلين (Actors) وحالات الاستخدام والعلاقات بين المعياريتين: «include» (تضمين إجباري) و«extend» (توسعة اختيارية/شرطية).



شكل 7-1: مخطط حالات الاستخدام مع علاقات «include» و«extend».

علاقة «extend» (توسعة)

سلوك اختياري يُضاف عند تحقق شرط، مثال: إصدار الشهادة يوسع حل الاختبار (عند النجاح)، ومنح XP يوسع المشاهدة/الاختبار (عند الإكمال).

علاقة «include» (تضمين)

سلوك إجباري تشترك فيه عدّة حالات، مثال: التسجيل، المشاهدة، وحل الاختبار جميعها «تتضمن» التحقق من الهوية؛ ولا يكتمل «التسجيل في مقرر» مدفوع دون «الدفع».

8 أمن البيانات وسلامتها ونزاهتها

يطبق النظام دفاعاً متعدد الطبقات (Defense in Depth) يغطي المصادقة، الصلاحيات، حماية المدخلات، وسلامة البيانات.

المصادقة وإدارة الجلسات

- **Supabase Auth** لإدارة الحسابات والجلسات، إلى جانب رموز جلسات محلية (`session_token`) يتم التحقق منها عبر `validateSession`.
- **تجزئة كلمات المرور** باستخدام `bcrypt` (Salt Rounds) قبل التخزين — لا تُخزن كلمات المرور نصياً أبداً.
- **التحقق من البريد الإلكتروني** عبر رموز (Tokens) مجزأة ومحدودة الصلاحية (`EmailVerification`), `PasswordReset`).
- **حراسة المسارات** في `middleware.ts` : منع الوصول غير المصادق عليه إلى `/dashboard` وإعادة التوجيه.

الصلاحيات والتحكم بالوصول (RBAC)

- نظام أدوار متدرج: `ADMIN` `MODERATOR` `TEACHER` `STUDENT` `GUEST` `SUPER_ADMIN`
- جداول `Role` / `Permission` / `RolePermission` لربط الصلاحيات بالأدوار ديناميكياً.
- الدوال `hasPermission` , `hasRole` , و `checkPermissionOrThrow` تفرض التحقق قبل كل عملية حساسة.
- `guardAction` يجمع: التحقق من الجلسة + الدور + تأكيد البريد + اعتماد المعلم + التبريد الزمني (Cooldown).

حماية المدخلات والطلبات

- **التحقق بـ Zod** لكل المدخلات قبل معالجتها لمنع البيانات غير الصالحة.
- **Prisma ORM** يستخدم استعلامات مُعاقلة (Parameterized) تمنع حقن SQL بطبيعتها.
- **تحديد معدل الطلبات (Rate Limiting)** عبر `rate-limiter.ts` لصدّ الهجمات التكرارية (يعيد 429 عند التجاوز).

8 أمن البيانات وسلامتها ونزاهتها

تابع — رؤوس الأمان، سلامة البيانات، والنزاهة والتدقيق.

● رؤوس الأمان (Security Headers)

تطبق على مستوى `next.config.ts` لكل المسارات:

الرأس	القيمة / الغرض
X-Frame-Options	DENY — منع التضمين داخل إطارات (Clickjacking).
X-Content-Type-Options	nosniff — منع تخمين نوع المحتوى.
Strict-Transport-Security	فرض HTTPS لمدة سنة مع النطاقات الفرعية.
Referrer-Policy	strict-origin-when-cross-origin .
Permissions-Policy	تعطيل الكاميرا/الميكروفون/الموقع وتتبع FLoC.
X-XSS-Protection	حماية إضافية ضد هجمات XSS.

● سلامة البيانات (Data Integrity)

- قيود قاعدة البيانات: مفاتيح أساسية (UUID)، قيود التفرد (@unique) على البريد وكود الإحالة ورقم الشهادة، ومفاتيح مركبة فريدة (@@unique([userId, courseId])).
- المفاتيح الأجنبية وسياسات الحذف: onDelete: Cascade للحفاظ على الاتساق، و SetNull حيث يلزم منع فقدان السجلات.
- الفهارس (Indexes): على الحقول كثيرة الاستعلام (البريد، الدور، الحالة) لضمان الأداء والاتساق.
- المعاملات الذرية: استخدام prisma.\$transaction في العمليات المركبة (التسجيل، إنشاء طلب المعلم) لضمان مبدأ «الكل أو لا شيء».
- القيم المُعدّدة (Enums): تقييد الحقول لقيم صحيحة فقط (الأدوار، الحالات، مستويات الصعوبة).

● النزاهة والتدقيق (Auditability)

- سجل التدقيق (AuditLog): تسجيل الأحداث الحساسة (تسجيل، تعديل صلاحيات) مع عنوان IP ووكيل المستخدم.
- سجلات المعاملات الاقتصادية (Transaction , XpTransaction) تُنشئ أثراً غير قابل للتلاعب لكل تغيير في النقاط والعملات.
- فحص الصحة عبر /api/health لمراقبة حالة الخادم وقاعدة البيانات، مع جاهزية للربط بـ Sentry.

خلاصة: يجمع النظام بين المصادقة القوية، والصلاحيات الدقيقة، والتحقق الصارم من المدخلات، وقيود قاعدة البيانات، والمعاملات الذرية، وسجلات التدقيق — لتحقيق السرية والسلامة والتوافر (CIA Triad).

